

Linear Algebra for Machine Learning

Revision notes built for understanding, not memorizing

1 Why Linear Algebra matters for ML

Every piece of data an ML model touches is secretly a list of numbers: a row of features, a pixel grid, a sentence turned into embeddings. A list of numbers is a vector, and a table of numbers is a matrix. So the moment you organize data for a model, you are already doing linear algebra, whether you notice it or not.

Linear algebra is the language for describing how these lists and tables of numbers combine, transform, and relate to each other. A neural network layer is a matrix multiplication. A word embedding is a vector. Image compression, recommendation systems, and search engines all lean on the exact same handful of ideas covered in this note.

Why this note exists

Most people learn linear algebra as abstract rules on paper and never connect it to what a model is actually doing. Every section below answers one question first: what real problem does this idea solve, before showing the formula.

2 Scalars, Vectors, Matrices and Tensors

These four words describe containers that hold different amounts of structure. Getting this hierarchy straight early makes everything later much easier to follow.

Object	What it holds	Example
Scalar	a single number	temperature = 36.6
Vector	a 1D list of numbers	a student's [height, weight, age]
Matrix	a 2D grid of numbers	a spreadsheet of students x features

Object	What it holds	Example
Tensor	any number of dimensions	a batch of color images (batch, height, width, channels)

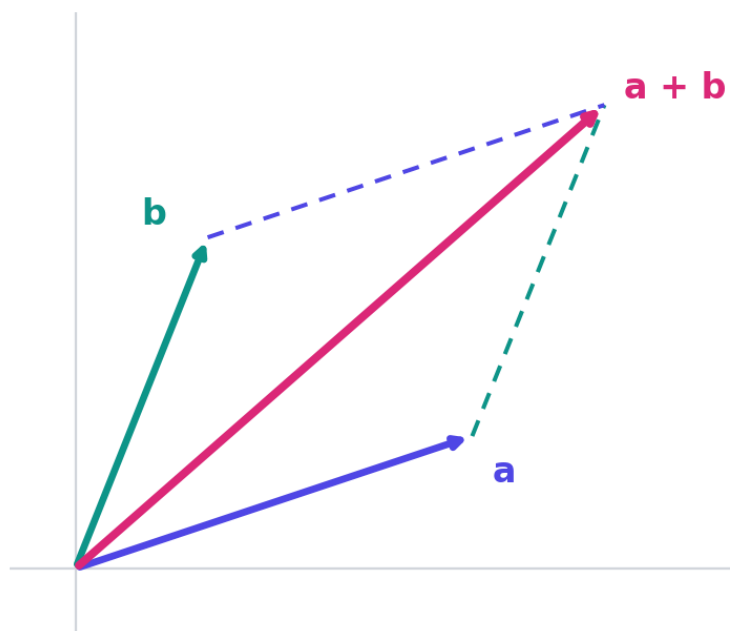
A vector is really just one row (or one column) of a matrix, and a matrix is really just a stack of vectors. A tensor simply keeps generalizing this stacking idea to as many dimensions as your problem needs. This is exactly why frameworks like PyTorch and TensorFlow are named after tensors, every single number that flows through a deep network, at every layer, is packaged as a tensor.

3 Vector Operations: Addition and Scaling

A vector is both a list of numbers and an arrow pointing from the origin to a location in space. Thinking of it as an arrow is what makes the operations below click intuitively instead of just being rules to memorize.

Vector Addition

Adding two vectors means placing one arrow at the tip of the other and drawing a new arrow from the very start to the very end. This is called the parallelogram rule, and it matches something you already know instinctively: if you walk 3 steps east then 2 steps north, your final position is the same regardless of which direction you did first.



$a + b$: walk along a , then walk along b , land at the same spot as walking the diagonal.

Scalar Multiplication

Multiplying a vector by a plain number just stretches or shrinks that same arrow, without changing which direction it points (unless the number is negative, which flips it backward).

Formulas

$$\mathbf{a} + \mathbf{b} = [\mathbf{a}_1 + \mathbf{b}_1, \mathbf{a}_2 + \mathbf{b}_2, \dots] \quad \mathbf{k} \times \mathbf{a} = [\mathbf{k} \cdot \mathbf{a}_1, \mathbf{k} \cdot \mathbf{a}_2, \dots]$$

Why this matters for ML: every single weight update during training is a vector operation exactly like this, add a scaled version of the gradient vector to the current weight vector to get the new weight vector.

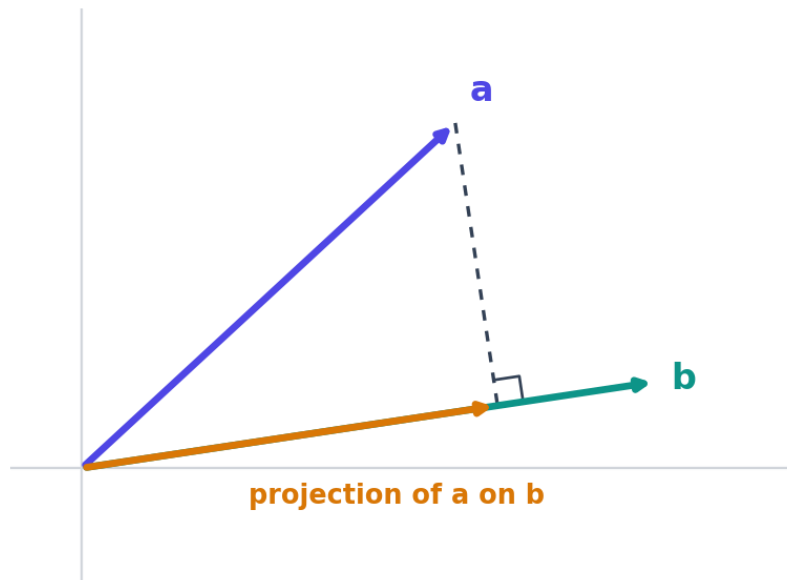
4 Dot Product

The dot product answers a very specific and very useful question: how much do two vectors point in the same direction. It squashes two vectors down into a single number that captures their alignment.

Formula

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}_1 \cdot \mathbf{b}_1 + \mathbf{a}_2 \cdot \mathbf{b}_2 + \dots + \mathbf{a}_n \cdot \mathbf{b}_n = |\mathbf{a}| |\mathbf{b}| \cos(\theta)$$

theta is the angle between the two vectors.



The dot product is (in effect) how far vector a reaches along the direction of vector b.

Why we need a whole separate concept here: addition tells you nothing about direction agreement, and just looking at two lists of numbers side by side does not tell you if they represent similar things. The dot product is the cleanest way to turn 'how similar are these two directions' into one comparable number.

- **If the dot product is large and positive**, the vectors point in a similar direction.
- **If it is near zero**, the vectors are roughly perpendicular, unrelated in direction.

- **If it is negative**, the vectors point in mostly opposite directions.

Where you will actually see this in ML

Cosine similarity, used constantly to compare word or document embeddings, is just a dot product with the lengths divided out. And a single artificial neuron's core computation, before the activation function, is nothing but a dot product between the input vector and the weight vector.

5 Vector Norms

A norm measures the length or size of a vector, turning a whole list of numbers into one single number that tells you how big that vector is overall. Different norms disagree on what counts as big in an interesting and useful way.

Norm	Formula	Intuition	Used for
L1 (Manhattan)	sum of $ x_i $	distance if you can only move along grid lines	Lasso regularization, encourages exact zeros
L2 (Euclidean)	$\sqrt{\text{sum of } x_i^2}$	straight line distance, the ordinary sense of length	Ridge regularization, weight decay
L-infinity	max of $ x_i $	only the single largest coordinate matters	bounding worst-case error

Why does ML need more than one definition of length? Because each one punishes largeness differently. L2 squares things, so it hates one giant value far more than several medium ones, which tends to shrink all weights a little. L1 grows in a straight line, so it happily pushes small, unimportant weights all the way down to exactly zero, which is exactly why L1 regularization is used to automatically select features by deleting the useless ones.

6 Matrices as Transformations

The biggest mental shift in linear algebra is this: a matrix is not just a grid of numbers sitting still, it is an instruction for moving every point in space somewhere else. Multiplying a vector by a matrix takes that vector's arrow and moves its tip to a brand new location, all in one operation.



The same grid of points, before and after being multiplied by a matrix. Straight lines stay straight, but everything stretches and tilts.

Notice the grid lines stay straight and evenly spaced, they never curve or bunch up unevenly, that straight-line-preserving property is exactly what makes something linear, and it is the entire reason this branch of math is called linear algebra.

Where you will actually see this in ML

Every fully connected layer in a neural network is exactly this: take the input vector, multiply it by a weight matrix, and land at a new vector in a different space. Stacking layers is stacking transformations, one grid-warp after another.

7 Matrix Multiplication

If a single matrix is one transformation, multiplying two matrices together builds a combined transformation, apply the second matrix's warp directly on top of the first one's warp, all in a single new matrix.

Rule

$(A \times B)$ has entry (i,j) = row i of A , dotted with column j of B

For $A(m \times n)$ times $B(n \times p)$, the inner dimensions n must match, and the result is size $m \times p$.

This is really just the dot product from section 4, run once for every row-column pairing. Order matters here, A times B is generally not the same as B times A , because applying warp A then warp B is usually not the same journey as applying B then A .

Where you will actually see this in ML

Passing a whole batch of examples through a neural network layer in one shot is exactly one matrix multiplication, the input batch matrix times the weight matrix. This is also exactly why GPUs,

which are built to crunch huge matrix multiplications in parallel, are what actually makes deep learning practical at scale.

8 Identity and Inverse Matrix

Identity Matrix

The identity matrix is the 'do nothing' transformation, multiplying any vector by it gives back that exact same vector untouched. It plays the same role that the number 1 plays in ordinary multiplication.

Notation

$$I \times v = v \text{ for any vector } v$$

I has 1s down the diagonal and 0s everywhere else.

Inverse Matrix

If a matrix A warps space one way, its inverse, written A^{-1} , is the exact matrix that un-warps it back to where it started. Multiplying A by its own inverse gives you back the identity matrix, the do-nothing transformation.

Notation

$$A \times A^{-1} = I$$

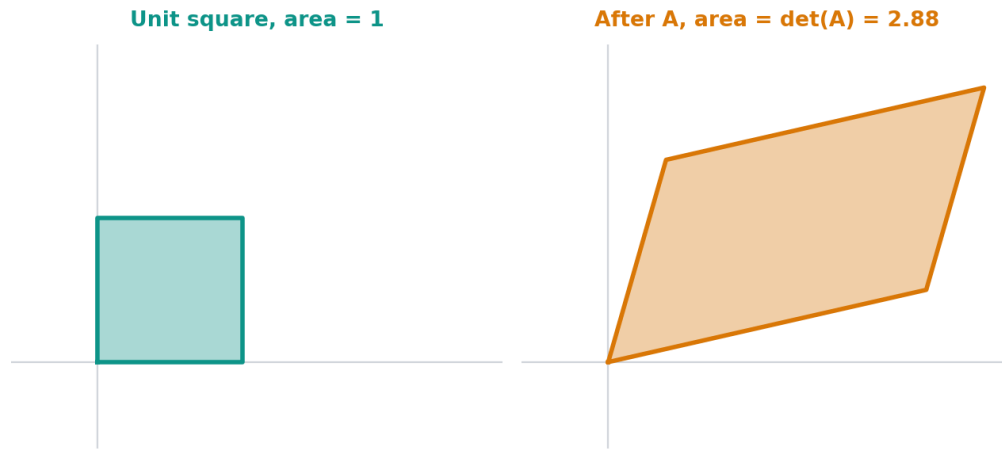
Not every matrix has an inverse. A matrix that squashes space down (collapses a dimension entirely) destroys information permanently, and there is no way to un-squash it back, so it has no inverse. Such a matrix is called singular.

Why we need this

Solving a system of linear equations, section 10, is really just asking 'what input produced this output', and the inverse matrix is the mathematical tool that un-does the transformation to answer exactly that.

9 Determinant

The determinant is a single number attached to a matrix that tells you how much that matrix's transformation stretches or squashes area (in 2D) or volume (in 3D and beyond).



A unit square has area 1. After the transformation, its new area equals the determinant of the matrix.

2x2 formula

$$\det\left(\begin{bmatrix} a & b \\ c & d \end{bmatrix}\right) = a.d - b.c$$

- **Determinant bigger than 1:** the transformation stretches space out, area grows.
- **Determinant between 0 and 1:** the transformation squashes space, area shrinks.
- **Determinant equal to 0:** the transformation crushes space down into a lower dimension, all the area disappears. This is exactly the singular matrix case from section 8, a matrix with determinant zero has no inverse, because you cannot un-crush something back into the information it lost.
- **Negative determinant:** the transformation also flips space, like a mirror image, in addition to scaling it.

10 Systems of Linear Equations

A huge number of real problems boil down to a set of equations with several unknowns that all need to be true simultaneously. Writing them out one by one gets messy fast, so linear algebra packages the whole system into one clean matrix equation.

The system

$$\mathbf{A} \mathbf{x} = \mathbf{b}$$

A is the matrix of known coefficients, x is the vector of unknowns you want to solve for, b is the vector of known results.

If A has an inverse, you can solve for the unknowns directly by undoing the transformation A applied: $\mathbf{x} = \mathbf{A}^{-1} \mathbf{b}$.

This single trick is exactly the backbone of Linear Regression, where solving for the best-fit coefficients means solving a system that looks exactly like this.

11 Linear Independence, Span and Rank

Span

The span of a set of vectors is every single location you could possibly reach by combining them (scaling and adding them together in any mix). Two vectors pointing in genuinely different directions in 2D can reach every single point on the plane.

Linear Independence

A set of vectors is linearly independent if none of them is just a scaled combination of the others, meaning every single one is pulling its own unique weight, adding real new reachable territory instead of overlapping with what the others already cover.

- **Example:** $[1,0]$ and $[0,1]$ are independent, together they reach all of 2D space. But $[1,0]$ and $[2,0]$ are dependent, they only ever point along the same line, no matter how you combine them you can never reach anywhere off that line.

Rank

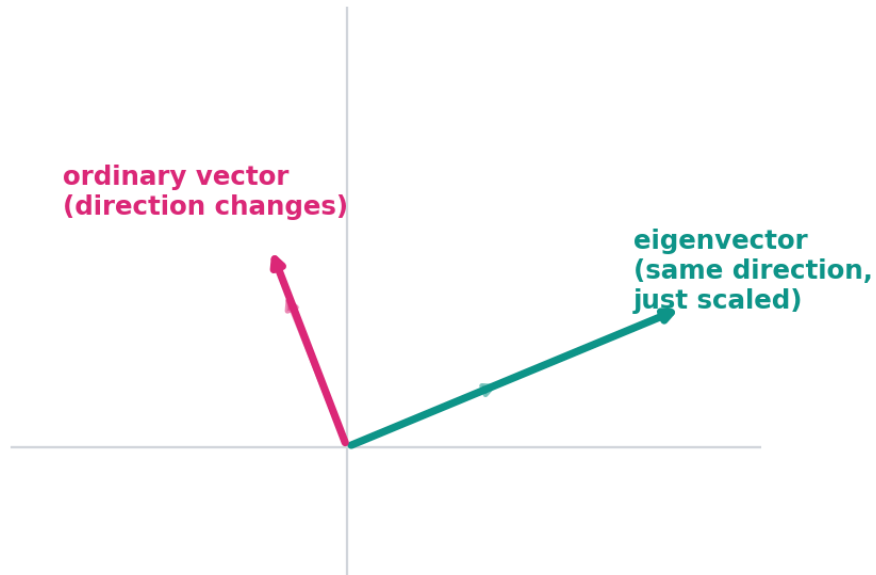
The rank of a matrix is simply how many linearly independent directions it actually contains, in other words, how many dimensions worth of genuinely new, non-redundant information it holds.

Why this matters for ML

If a dataset's feature matrix has low rank, some columns are redundant combinations of others, quietly telling the model nothing new while adding noise and instability. Checking rank is a quick way to catch duplicate or redundant features before they cause problems during training.

12 Eigenvalues and Eigenvectors

Every matrix transformation grabs and twists space around, most vectors get rotated onto a completely new direction. But for almost every matrix, there exist a few special vectors that refuse to change direction, the transformation only stretches or shrinks them along the exact same line they already pointed on. Those are eigenvectors, and the amount they get stretched by is the corresponding eigenvalue.



An ordinary vector gets rotated onto a new direction, but the eigenvector stays on its original line, only its length changes.

Definition

$$A v = \lambda v$$

v is the eigenvector, λ is the eigenvalue, the plain number telling you how much v got stretched (or shrunk, or flipped if negative).

Why this matters so much: an eigenvector reveals a direction where a matrix's behavior is as simple as ordinary multiplication by a single number, instead of a full space-warping mess. Finding these directions is exactly how you find the natural axes along which a transformation, or a dataset, actually behaves, which leads straight into PCA in section 15.

13 Eigendecomposition and Diagonalization

Once you have found all of a matrix's eigenvectors and eigenvalues, you can rewrite the entire original matrix as a combination of just those pieces. This is useful because working with the pieces (especially the simple, purely diagonal eigenvalue matrix) is often far easier than working with the original tangled matrix.

Formula

$$A = V \text{diag}(\lambda) V^{-1}$$

V is the matrix whose columns are the eigenvectors, $\text{diag}(\lambda)$ is a matrix with the eigenvalues down the diagonal and zeros elsewhere.

A very practical payoff: raising a matrix to a large power (needed in some ML algorithms and in analyzing repeated processes) becomes trivial once diagonalized, since raising a diagonal matrix to a power just means raising each eigenvalue on the diagonal to that power, instead of painfully multiplying the full matrix by itself over and over.

14 Singular Value Decomposition (SVD)

Eigendecomposition has an annoying restriction, it only cleanly works on square matrices. Real data is almost never conveniently square, you might have 10,000 rows of customers and only 20 columns of features. SVD is the generalization that works on any matrix, of any shape, no exceptions.

Formula

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

U and V are made of orthogonal (mutually perpendicular) direction vectors, Sigma is a diagonal matrix of singular values, ranked from most to least important, describing how much each direction matters.

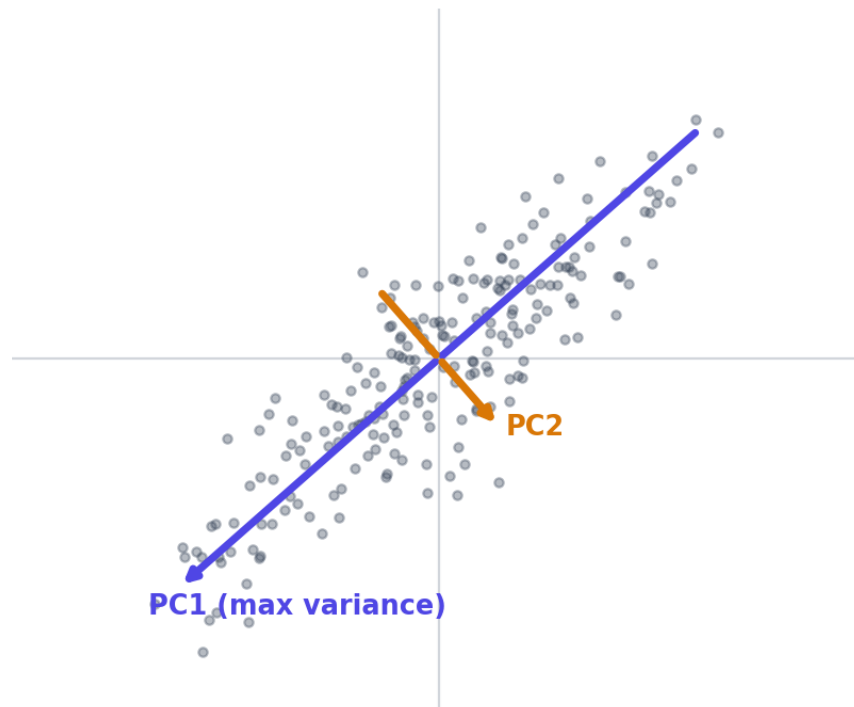
The intuition to hold onto: SVD splits any transformation into three honest steps, rotate, stretch along clean perpendicular axes, then rotate again. Because the singular values in Sigma are ranked by importance, you can throw away the smallest ones and keep only the biggest handful, getting a compressed approximation of the original matrix that keeps almost all of the important information.

Where you will actually see this in ML

Recommendation systems (like early Netflix-style matrix factorization), image compression, noise reduction, and topic modelling in text (Latent Semantic Analysis) are all direct real-world uses of SVD's ability to compress a big matrix down to its most important few directions.

15 Principal Component Analysis (PCA)

Real datasets often have many columns that are secretly correlated and repeating similar information sideways. PCA answers a direct question: what is the smallest number of new, made-up directions I can use to capture almost all of the meaningful variation in this data.



A cloud of correlated data points. PC1 points along the direction of maximum spread, PC2 is perpendicular to it and captures whatever spread is left.

Here is exactly how the pieces from earlier sections snap together: PCA calculates the covariance matrix of your data (capturing how every feature relates to every other feature), then finds that matrix's eigenvectors. The eigenvector with the largest eigenvalue points along the direction where your data spreads out the most, that becomes your first principal component. The next eigenvector, perpendicular to the first, captures the next largest remaining spread, and so on.

Why we bother doing this at all: fewer, smarter dimensions mean faster training, less overfitting risk, easier visualization (squashing hundreds of columns down to just 2 or 3 for plotting), and removing redundant, correlated noise from your features.

16 Orthogonality

Two vectors are orthogonal if they sit at a perfect right angle to each other, which, going back to section 4, means their dot product is exactly zero. Orthogonal directions are valuable because they capture completely independent information, one direction tells you nothing at all about the other.

An orthogonal matrix is a matrix built entirely out of directions that are all mutually perpendicular to each other and each exactly length 1. Multiplying by an orthogonal matrix only ever rotates or reflects space, it never stretches or squashes it, which also means its inverse is trivially easy to compute, it is just its own transpose.

Property

if Q is orthogonal, then $Q^{-1} = Q^T$

Flipping rows into columns instantly gives you the inverse, no heavy computation needed.

Why this matters

SVD's U and V matrices in section 14 are both orthogonal on purpose, that is exactly what guarantees the rotate-stretch-rotate story holds together cleanly, with no hidden extra shearing or distortion sneaking into the picture.

17 Tensors in Deep Learning

A tensor is simply the natural extension of scalar, vector and matrix to any number of dimensions at once, and deep learning leans on this constantly because real world data rarely fits neatly into a flat 2D table.

Data	Typical tensor shape
A single grayscale image	height x width
A single color image	height x width x color channels
A batch of color images	batch size x height x width x channels
A batch of sentences (word embeddings)	batch size x sequence length x embedding size

Every operation covered in this note, addition, dot products, matrix multiplication, all of it, still applies, just stretched across more dimensions at once. A convolution in a CNN, for instance, is really a structured, repeated matrix multiplication sliding across a tensor.

18 Where it all fits: the ML map

A quick reference tying every idea in this note directly back to where it shows up in an actual ML workflow.

Concept	Shows up in ML as
Vector	a single data point, a word embedding, a weight vector
Matrix	a dataset (rows x features), a neural network layer's weights
Dot product	cosine similarity, a single neuron's core computation
Norms (L_1 , L_2)	Lasso and Ridge regularization, weight decay

Concept	Shows up in ML as
Matrix multiplication	passing a batch through a neural network layer
Inverse and $Ax=b$	solving Linear Regression's closed form solution
Determinant	checking if a matrix is invertible, singular, safe to use
Rank	detecting redundant or duplicate features
Eigenvectors / Eigenvalues	finding a transformation's natural, simplest axes
SVD	recommendation systems, compression, topic modelling
PCA	dimensionality reduction, visualization, denoising
Orthogonal matrices	stable, distortion free rotations inside SVD
Tensors	how every framework (PyTorch, TensorFlow) stores all data

End of notes. If short on time before revision, prioritize dot product, matrix transformations, eigenvectors, and PCA, these four ideas connect almost everything else here to a real ML use case.